



Case4



Guide de Code Review — Feuille à remplir

| Objectif : utiliser cette feuille comme un *fil conducteur* pendant l'exercice.

| Remplis les zones "🔧 Réponse" au fur et à mesure.



Étape 1 — Cadrage (Top-Down / Haut Niveau)

1) Intention Métier

Question : À quoi sert ce fichier ? Quel problème résout-il ?

🔧 **Réponse :**

- Contexte / produit : api web async
- Problème utilisateur / besoin : recherche des documents, orc_process de docs
- Entrées principales : requête utilisateur
- Sorties principales : le document avec extraction ocr

2) Architecture Globale

Question : Comment le code est-il découpé ? (classes, services, endpoints, fonctions utilitaires)

🔧 **Réponse :**

- Composants / modules : Classe DocumentBatchRepository → connection a db, recupère des docs en fct de leurs ids., Classe ExternalIndexingClient → essaye une connexion au search_engine avec réessais. Fonction utilitaire heavy_ocr_process, 2 endpoints fast api

- Point d'entrée (main / handler / endpoint principal) :

"/batch_process" post → reçoit une requête, renvoie les documents demandés

"/process_ocr_pdf" post → faire du ocr process sur un document de la bdd

- Données qui circulent (schéma rapide) :

requête utilisateur → liste de ids de documents → documents + connexion a une indexation externe → reponse

requête → ocr_process du doc → renvoi du texte extrait.

3) Périmètre & Dépendances

Question : Quels sont les modules externes importés et les flux d'entrées/sorties ?

 **Réponse :**

- Dépendances externes (librairies / services) :
 - time, requests, logging, typing, fastapi
 - Dépendances internes (fichiers / packages du repo) :
 - document pdf
 - I/O (réseau, DB, fichiers, cache, queue, etc.) :
 - data base
 - search engine interne
 - Zones sensibles (auth, secrets, PII, permissions) :
 - lien vers le search engine
-

Étape 2 — Inspection par les 4 Piliers (Scan ligne par ligne)

| Pour chaque point, note **ce que tu observes** et **les anomalies**.

| (Tu peux référencer des lignes/fonctions : `foo()` , `MyClass.method` , "ligne ~120", etc.)

Pilier 1 — Logique & Concurrency

Checklist mentale :

- Le flux d'exécution est-il correct ?
- Async / threads / événements : pas de blocage, pas de race conditions, pas d'await oubliés ?

Notes / anomalies :

- def fetch_documents_by_ids → async
 - def sync_to_search_engine_with_retry → async
 -
-

Pilier 2 — Gestion des Ressources & Mémoire

Checklist mentale :

- Pas de fuite d'état global, objets bien isolés ?
- Connexions / fichiers / sockets : cycle de vie propre (ouverture/fermeture garanties) ?

Notes / anomalies :

- response = requests.post(url, json=payload, timeout=5) → risque de créer des doublons ! utiliser un idempotent id a chaque post
 -
 -
-

Pilier 3 — Robustesse & Sécurité

Checklist mentale :

- Erreurs / exceptions : traitées correctement (loggées, relancées, pas masquées) ?
- Inputs : validés (null/absent/format), pas d'injection, pas de confiance implicite ?

Notes / anomalies :

- self.db.execute("SELECT id, title, content FROM documents WHERE id = %s", (doc_id,)) → pas de try/except si appel raté
- url = "<https://search-engine.internal/api/v1/index>" → secret management

- request: Request → pydantic
- if not success: → return si erreur dans le endpoint → validation success
200 faux → utiliser raise error

Pilier 4 — Architecture, Design & Performance

Checklist mentale :

- SRP : une seule responsabilité par fonction/classe ?
- Traitements lourds inutiles (boucles, N+1, requêtes multiples, allocations) ?
- Nommage clair, pas ambigu ?



Notes / anomalies :

- for doc_id in document_ids: → N appels réseau au lieu d'un groupé
- execute_heavy_ocr_processing → mettre en background si utilisé, et donner un feedback progression a l'utilisateur
-



Étape 3 — Feedback Structuré (SÉVÉRITÉ / IMPACT / SOLUTION)

A) Rappel des tags de sévérité

- **[BLOCKER]** : crash fatal, bug logique critique, faille de sécurité majeure
- **[MAJOR]** : perf sous charge, fuite de ressources, mauvaise architecture
- **[MINOR]** : lisibilité, typage manquant, refactoring local
- **[NIT]** : cosmétique / style

B) Commentaires de review (à remplir)

Copie-colle ce bloc autant de fois que nécessaire :

- [TAG_SÉVÉRITÉ] Emplacement / Fonction :
 - Problème :
 - Impact (en prod) :
 - Solution (proposition concrète, idéalement avec pseudo-code / code) :

Commentaire 1 :

- [____] `fetch_documents_by_ids` :
 - Problème : pas async sur fonction appel reseau
 - Impact : bloque le code
 - Solution : async et await l'appel db

Commentaire 2 :

- [____] `sync_to_search_engine_with_retry` :
 - Problème : pareil
 - Impact :
 - Solution :

Commentaire 3 :

- [____] `response = requests.post(url, json=payload, timeout=5):`
 - Problème : post sur le serveur sans idempotent id
 - Impact : risque de doublons
 - Solution : ajouter un idempotent id

Commentaire 3 :

- [____] `self.db.execute("SELECT id, title...):`
 - Problème : pas de try/except alors que appel réseau bdd
 - Impact : si erreur aucun moyen de savoir + crash
 - Solution : faire un try/except

Commentaire 3 :

- [____] `url = "https://search-engine.internal/api/v1/index":`
 - Problème : lien en dur
 - Impact : data leakage
 - Solution : `os.environ[lien]`

Commentaire 3 :

- [____] `request: Request`
 - Problème : pas de vérification du format de la requête

- Impact : extraction de feature risquée
- Solution : pydantic

Commentaire 3 :

- [____] _____ :
 - Problème :
 - Impact :
 - Solution :

Commentaire 3 :

- [____] if not success: → return :
 - Problème : return en cas de problème dans un endpoint api
 - Impact : retourne un succès a fastapi alors que non
 - Solution : utilise raise HTTPException

Commentaire 3 :

- [____] for doc_id in document_ids:
 - Problème : un appel database par document
 - Impact : gaspillage d'appels réseau
 - Solution : 1 appel plus query sql groupée suffit

Commentaire 3 :

- [____] execute_heavy_ocr_processing :
 - Problème : fonction lourde bloquante synchrone
 - Impact : lantence de zinzin bloque l'endpoint pour tout le monde
 - Solution : mettre en background task et donner feedback progression a l'utilisateur

Commentaire 3 :

- [____] supplémentaire:
 - Problème : aucun log + trace_id
 - Impact : impossible de remonter les problèmes / historique
 - Solution : remplir logger en cas d'erreur, et utiliser uuid.uuid4()